Rynix Programming Language

1. Language Overview

1.1 Introduction

Rynix is a programming language designed to support procedural programming, object-oriented programming, and structured software development. It combines the clean syntax style of Python with the strong typing and structured programming concepts found in Kotlin. Rynix programs are written as sequences of declarations, statements, expressions, functions, and optional class definitions.

---

2. General Language Rules

2.1 Source File

A Rynix source file contains the complete program written using Rynix syntax.
A program may contain:

- Variable declarations
- Constant declarations
- Function definitions
- Class definitions
- Executable statements

A Rynix source file normally starts execution from a function named `main`.
Example:

```
fun main(): Void {
    print("Hello Rynix")
}
```

2.2 Case Sensitivity

Rynix is case-sensitive. This means names with different capitalization are considered different.
Example:

```
var age: Int = 20
var Age: Int = 30
```

`age` and `Age` are two different variables.

2.3 Comments

Comments are text written inside the program that is ignored during execution. Comments are used to explain code or temporarily disable parts of a program.
Rynix supports:

- **Single-line comments**
  A single-line comment begins with `//` and continues until the end of the line.

Example:

```
// Store the user age
var age: Int = 25
```

- **Multi-line comments**
  A multi-line comment begins with `/*` and ends with `*/`.
  Example:

```
/*
This program calculates
the final result
*/
var result: Int = 100
```

---

3. Identifiers

Description

An identifier is a name used to identify program elements such as:

- Variables
- Constants
- Functions
- Classes

Identifiers should describe the purpose of the element they represent.
Examples:

```
studentName
totalPrice
calculateAverage
```

Identifier Rules

An identifier:

- Must begin with a letter or underscore
- Can contain letters, numbers, and underscores
- Cannot contain spaces
- Cannot be a reserved Rynix keyword

Valid	Invalid

`user1`
`_total`
`studentAge`
	`1user`
	`student age`
	`fun`

---

4. Reserved Words

Reserved words are special words that have predefined meanings in Rynix and

cannot be used as identifiers.
Rynix reserved words include:
`var, val, fun, class, if, else, while, for, in, return, true, false, null, new, print, input, break, continue`
Example:
`var fun: Int = 5` // Invalid because `fun` is reserved.

---

5. Data Types

Rynix is a strongly typed language. Every value has a specific type that determines what operations can be performed on it.

5.1 Integer Type

Description

`Int` stores whole numbers without decimal values.
Examples:

```
var age: Int = 20
var count: Int = -5
```

5.2 Floating Point Type

Description

`Float` stores numbers containing decimal values.
Example:

```
var price: Float = 15.75
```

5.3 String Type

Description

`String` stores a sequence of characters surrounded by double quotes.
Example:

```
var name: String = "Ahmed"
```

Strings can contain:

- Letters
- Numbers
- Symbols
- Spaces

Example:

```
var message: String = "Hello Rynix"
```

5.4 Boolean Type

Description

`Bool` represents logical values. A Boolean value can only be:

`true`

`false`

Example:

```
var finished: Bool = false
```

Boolean values are mainly used with conditions.

Example:

```
if finished {
    print("Done")
}
```

5.5 Character Type

Description

`Char` stores a single character.

Example:

```
var grade: Char = 'A'
```

A character must use single quotes.

5.6 Array Type

Description

An array stores multiple values of the same type in an ordered collection.

Example:

```
var numbers: Array<Int> = [1,2,3,4]
```

Array elements are accessed using an index.

Example:

```
print(numbers[0])
```

The first element has index zero.

---

6. Variables

Description

A variable is a storage location whose value can change during program execution.

Variables are created using the keyword `var`.

Example:

```
var score: Int = 0
score = 10
```

A variable must have:

- A name
- A type
- An initial value

---

7. Constants

Description

A constant is a value that cannot be changed after initialization.
Constants are created using `val`.
Example:

```
val maxScore: Int = 100
maxScore = 200 //
```

Invalid because constants cannot be modified.

---

8. Operators

Description

Operators are symbols used to perform operations on values.

Arithmetic Operators

Addition (+), Subtraction (-), Multiplication (*), Division (/), Modulus (%)
Example:

```
var sum = a + b
x = x - 1
total = price * quantity
average = total / count
result = 10 % 3
```

---

9. If Statement

Description

The `if` statement allows a program to execute code only when a condition is true.
Structure:

```
if condition {
    // statements
}
```

Example:

```
if age >= 18 {
    print("Adult")
}
```

10. Else Statement

Description

The `else` statement provides an alternative block that executes when the `if` condition is false.

Example:

```
if grade >= 50 {
    print("Pass")
}
else {
    print("Fail")
}
```

11. While Loop

Description

The `while` loop repeatedly executes a block of statements as long as a condition remains true.

A while loop is useful when the number of repetitions is unknown before execution.

Structure:

```
while condition {
   // statements
}
```

Example:

```
var counter: Int = 1
while counter <= 5 {
    print(counter)
    counter = counter + 1
}
```

Example:

```
while x < 100 {
    if x == 50 {
        break
    }
    x = x + 1
}
```

12. For Loop

Description

The `for` loop repeats a block of code for every value inside a range or collection.

Example:

```
for i in 1..10 {
    print(i)
}
```

The loop variable receives each value in the range.

Example:

```
for item in numbers {
    print(item)
}
```

---

13. Functions

Description

A function is a reusable block of code that performs a specific task.

Functions may:

- Receive input values
- Perform operations
- Return a result

Example:

```
fun multiply(a: Int, b: Int): Int {
    return a * b
}
```

Calling:

```
var result = multiply(5,3)
```

---

14. Return Statement

Description

The `return` statement sends a value from a function back to the caller.

Example:

```
fun square(x: Int): Int {
    return x * x
}
```

Functions with no returned value use:

```
Void
```

Example:

```
fun hello(): Void {
    print("Hello")
}
```

---

15. Function Parameters

Description

Parameters are variables that receive values from the code that calls a function.
Parameters are defined inside the parentheses after the function name.
Example:

```
fun greet(name: String): Void {
    print(name)
}
```

Calling the function:

```
greet("Ahmed")
```

A function can receive multiple parameters separated by commas.
Example:

```
fun calculateTotal(price: Float, quantity: Int): Float {
    return price * quantity
}
```

Calling:

```
var result: Float = calculateTotal(10.5, 3)
```

---

16. Function Calls

Description

A function call executes the instructions inside a previously defined function.
A function call consists of:

- Function name
- Parentheses
- Optional arguments

Example:

```
print("Hello")
```

Another example:

```
var answer: Int = add(5,10)
```

---

17. Arrays

Description

An array stores multiple values of the same type. Arrays maintain the order of their elements. An array is created using square brackets.
Example:

```
var numbers: Array<Int> = [10,20,30]
```

The array contains:

```
Index    Value
0        10
```

```
1        20
2        30
```

Example:

```
var names: Array<String> = ["Ali","Sara","Omar"]
```

Individual elements are accessed using their index.

Example:

```
print(names[1])
```

Output:

```
Sara
```

An array element can be changed using assignment.

Example:

```
numbers[0] = 100
```

Before:

```
[10,20,30]
```

After:

```
[100,20,30]
```

17.1 Array Length

Rynix provides a built-in property called `length`.

Example:

```
var size: Int = numbers.length
```

17.2 Array Looping

Arrays can be processed using a `for` loop.

Example:

```
for item in numbers {
    print(item)
}
```

---

18. Classes

Description

A class defines a structure that combines data and behavior. A class can contain:

- Variables
- Functions

A class represents a blueprint for creating objects.

Example:

```
class Student {
    var name: String
    var age: Int
}
```

---

19. Objects

Description

An object is an instance created from a class.
Objects store their own values and can access class functions.
Example:

```
var student: Student = new Student()
```

---

20. Object Properties

Description

Properties are variables stored inside an object.
Example:

```
student.name = "Ahmed"
student.age = 20
```

---

21. Class Functions

Description

Functions inside a class are called methods.
Example:

```
class Student {
    var name: String
    fun display(): Void {
        print(name)
    }
}
```

Calling:

```
student.display()
```

---

22. Constructors

Description

A constructor initializes an object when it is created.
Rynix constructors use the keyword `init`.
Example:

```
class Student {
    var name: String
    var age: Int
    init(n: String, a: Int) {
        name = n
```

```
        age = a
    }
}
```
Creating an object:
```
var s = new Student("Ali",20)
```

---

23. Null Value

Description

`null` represents the absence of a value.
Example:
```
var name: String = null
```
A variable containing `null` does not point to an actual value.

---

24. Null Checking

Description

Before using a nullable value, the program should verify that it is not null.
Example:
```
if name != null {
    print(name)
}
```

---

25. Input and Output

Description

Rynix provides built-in functions for communicating with users.

25.1 Print Function

Description

`print()` displays information on the screen.
Example:
```
print("Hello Rynix")
```
Printing variables:
```
var x: Int = 10
print(x)
```

25.2 Input Function

Description

`input()` reads text entered by the user.
Example:
```
var name: String = input()
```
The returned value is stored in the variable.

---

26. Break Statement

Description

`break` immediately stops the nearest loop.
Example:
```
while true {
    if x == 10 {
        break
    }
}
```

---

27. Continue Statement

Description

`continue` skips the current loop iteration and moves to the next one.
Example:
```
for i in 1..5 {
    if i == 3 {
        continue
    }
    print(i)
}
```
Output:
```
1
2
4
5
```

---

28. Scope

Scope defines where a variable can be accessed. Rynixhas two main scopes:

- Global scope
- Local scope

28.1 Local Variables

A variable declared inside a block exists only inside that block.

Example:

```
if true {
    var x: Int = 10
    print(x)
}
```

Outside the block: `print(x)` is invalid because `x` no longer exists.

28.2 Function Scope

Variables created inside functions are private to that function.
Example:

```
fun test(): Void {
    var number: Int = 5
}
```

`number` cannot be used outside `test`.

---

29. Complete Rynix Program Examples

The following examples demonstrate different language features.

Example 1: Calculator Program

```
fun add(a: Int, b: Int): Int {
    return a + b
}

fun subtract(a: Int, b: Int): Int {
    return a - b
}
fun main(): Void {
    var x: Int = 20
    var y: Int = 5
    var sum: Int = add(x,y)
    var difference: Int = subtract(x,y)
    print(sum)
    print(difference)
    if sum > difference {
        print("Addition is larger")
    }
    else {
        print("Subtraction is larger")
    }
}
```

Example 2: Student Management System

```
class Student {
    var name: String
    var grade: Int
    init(n: String, g: Int) {
        name = n
        grade = g
    }
    fun display(): Void {
        print(name)
        print(grade)
    }
}
fun main(): Void {
    var students: Array<Student>
    var s1 = new Student("Ali",90)
    var s2 = new Student("Sara",85)
    s1.display()
    s2.display()
    if s1.grade > s2.grade {
        print("Ali has higher grade")
    }
    else {
        print("Sara has higher grade")
    }
}
```

Example 3: Complete Rynix Program

```
class BankAccount {
    var owner: String
    var balance: Float
    init(name: String, amount: Float) {
        owner = name
        balance = amount
    }
    fun deposit(value: Float): Void {
        balance = balance + value
    }
    fun withdraw(value: Float): Bool {
        if value <= balance {
```

```
            balance = balance - value
            return true
        }
        return false
    }
}
fun main(): Void {
    print("Enter your name:")
    var name: String = input()
    var account = new BankAccount(name,1000.0)
    for i in 1..5 {
        account.deposit(100.0)
    }
    var success: Bool = account.withdraw(250.0)
    if success {
        print("Transaction completed")
    }
    else {
        print("Not enough balance")
    }
    while account.balance < 2000 {
        account.deposit(50.0)
    }
    print(account.balance)
}
```